

LAPORAN TUGAS BESAR CLO4



Kelompok Ayondi

Disusun oleh:

Muhammad Ayondi Ramadhan	103082400011
Raditya Prasna Putra D	103082400027
Putra Erlang Duawa	103082400023
Nandhana Atha Balangga	103082400025
Chrisel Arthalia Putri	1201210510

Mata Kuliah Konstruksi Perangkat Lunak Semester Genap Tahun Ajaran 2025-2026

**Program Studi S1 Rekayasa Perangkat Lunak Telkom
University
Direktorat Kampus Surabaya**

A. Deskripsi Aplikasi

Aplikasi Manajemen Apotek merupakan sistem informasi berbasis *desktop* yang dirancang untuk mengoptimalkan pengelolaan data obat, pemantauan stok, dan efisiensi transaksi penjualan di apotek. Sistem ini mengadopsi arsitektur *client-server* yang terintegrasi dengan API eksternal untuk memastikan sinkronisasi data yang *real-time* dan akurat.

Fitur utama aplikasi mencakup manajemen inventaris dengan klasifikasi status otomatis (Tersedia, Stok Rendah, Kadaluarasa), sistem kasir terpadu dengan kalkulasi pajak dan diskon yang fleksibel, hingga pembuatan struk belanja otomatis dalam format teks. Dengan antarmuka yang intuitif, aplikasi ini bertujuan untuk meminimalisir kesalahan manusia dalam pencatatan data medis dan mempercepat proses pelayanan kepada pelanggan.

B. Informasi Project

1. Link Repository Github:

[\[https://github.com/putraerlang2-glitch/TUBES-KPL.git\]](https://github.com/putraerlang2-glitch/TUBES-KPL.git)

2. Teknologi: C# .NET Framework 4.7.2, Windows Forms, .NET 6 Web API.

3. Fitur Login

4. Fitur CRUD Data Obat

5. Fitur Pencarian

6. Fitur Auto-Update Status Obat

7. Fitur Transaksi

C. Pembagian Tugas

Muhammad Ayondi Ramadhan	103082400011	Teknik Konstruksi: Code Reuse & Table-driven Tanggung Jawab: - Fitur Laporan Tamu
Raditya Prasna Putra Dharmawan	103082400027	Teknik Konstruksi: Generics & Code Reuse Tanggung Jawab: - Fitur Tambah
Nandana Atha Balangga	103082400025	Teknik Konstruksi: Code Reuse & Runtime Configuration Tanggung Jawab: - Fitur Pengaduan Fasilitas - Fitur Authentication
Putra Erlang Duawa	103082400023	Teknik Konstruksi: Table-driven & Generic Parameter Tanggung Jawab: - Fitur Transaksi - Fitur Cari Obat - Pembuatan Tabel Obat

Chrisel Artalia Putri	103082400027	Teknik Konstruksi: Table-driven & Runtime Configuration Tanggung Jawab: - Fitur Pengaduan Keamanan - Fitur Manajemen Pengguna
-----------------------	--------------	---

D. Penerapan Design Pattern

1. Singleton Pattern

Pada project ini kami telah menerapkan yaitu *Singleton Pattern* dan . *Singleton Pattern* sendiri bisa dibilang sebuah pola desain yang memastikan sebuah kelas memiliki satu objek atau contoh yang berisi data dan nantinya contoh tersebut bisa digunakan di fitur lain atau di seluruh fitur aplikasi. Penggunaan *Singleton pattern* sebagai design pattern bisa berguna untuk menghemat memori dan mengontrol sumber daya agar memori tidak boros.

Masalah yang diselesaikan:

Dalam sistem kasir apotek, nilai regulasi seperti persentase Pajak PPN dan Diskon adalah data global yang harus konsisten di seluruh bagian aplikasi. Jika nilai ini dideklarasikan berulang kali di setiap proses transaksi, akan terjadi inkonsistensi data jika admin mengubah nilainya di tengah jalan.

Keuntungan penerapan:

- Menyediakan satu sumber kebenaran data (*Single Source of Truth*).
- Mencegah instansiasi berulang untuk objek yang sifatnya konstan/global.

Untuk penerapan dari *singleton pattern* ini bisa dilihat pada halaman RuntimeConfig.cs. Jika dilihat dari potongan kode di bawah, class RuntimeConfig bisa dianggap sebagai aturan, dan aturan ini berhubungan dengan fitur yang menggunakan pajak dan juga diskon.

```
namespace TubesKPL
{
    // [LANGGA - RUNTIME CONFIGURATOR]
    17 references
    public static class RuntimeConfig
    {
        // Default Pajak PPN 11%
        7 references
        public static decimal PajakPPN { get; set; } = 0.11m;

        // Default Diskon 0%
        7 references
        public static decimal DiskonAktif { get; set; } = 0.00m;

        // Method untuk mengubah nilai saat aplikasi berjalan (Runtime)
        5 references
        public static void UpdateConfig(decimal pajakBaru, decimal diskonBaru)
        {
            PajakPPN = pajakBaru;
            DiskonAktif = diskonBaru;
        }
    }
}
```

Penerapan pada project:

```
private void BtnUbahConfig_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(TBPajak.Text) || string.IsNullOrEmpty(TBDiskon.Text))
    {
        MessageBox.Show("Kotak Pajak dan Diskon harus diisi angka dulu ya!", "Peringatan", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (decimal.TryParse(TBPajak.Text, out decimal pajakInput) &&
        decimal.TryParse(TBDiskon.Text, out decimal diskonInput))
    {
        decimal pajakDesimal = pajakInput / 100m;
        decimal diskonDesimal = diskonInput / 100m;

        RuntimeConfig.UpdateConfig(pajakDesimal, diskonDesimal);

        MessageBox.Show($"Mantap! Konfigurasi berhasil diubah secara Runtime.\nPajak Baru: {pajakInput}%\nDiskon Baru: {diskonInput}%\n\nSil",
        }
    }
    else
    {
        MessageBox.Show("Tolong masukkan format angka yang benar ya!", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Penerapan pertama terdapat pada button ubah config yang ada di halaman transaksi. Button ubah config sendiri memiliki tujuan untuk mengubah nilai pajak dan diskon berdasarkan inputan dari user. Ketika user menekan button ubah config, maka program akan menjalankan logika dari button ini dan menyimpan value diskon dan pajak terbaru ke singleton pattern.

```
// 2. Terapkan RUNTIME CONFIGURATOR
decimal nominalDiskon = subtotal * RuntimeConfig.DiskonAktif;
decimal subtotalSetelahDiskon = subtotal - nominalDiskon;
decimal nominalPajak = subtotalSetelahDiskon * RuntimeConfig.PajakPPN;
decimal grandTotal = subtotalSetelahDiskon + nominalPajak;
```

Kemudian untuk penerapan yang kedua, masih ada di halaman yang sama tetapi berada di logika button hitung (Checkout). Pada proses total pembayaran, aplikasi mengambil nilai diskon dan pajak dari RuntimeConfig. Karena Runtime Config menggunakan singleton pattern, maka nilai yang telah diubah oleh user dapat dipakai secara otomatis oleh seluruh proses transaksi.

```
foreach (var item in keranjang)
{
    sb.AppendLine(item.obat.nama);
    sb.AppendLine($"{item.jumlah} x Rp {item.obat.harga:N0} = Rp {item.Subtotal():N0}");
}

sb.AppendLine("-----");
sb.AppendLine($"Subtotal : Rp {subtotal:N0}");

decimal nominalDiskon = subtotal * diskon;
decimal nominalPajak = (subtotal - nominalDiskon) * pajak;

sb.AppendLine($"Diskon ({(diskon * 100):0}%) : - Rp {nominalDiskon:N0}");
sb.AppendLine($"Pajak PPN ({(pajak * 100):0}%) : + Rp {nominalPajak:N0}");
sb.AppendLine("-----");
sb.AppendLine($"TOTAL BAYAR : Rp {totalAkhir:N0}");
sb.AppendLine($"UANG DITERIMA : Rp {bayar:N0}");
sb.AppendLine($"KEMBALIAN : Rp {kembalian:N0}");
sb.AppendLine("=====");
sb.AppendLine($"TERIMA KASIH, SEMOGA LEKAS SEMBUH ");
sb.AppendLine("=====");

try
{
    string namaFile = $"Struk_{DateTime.Now:yyyyMMdd_HH:mm:ss}.txt";
    File.WriteAllText(namaFile, sb.ToString());
    MessageBox.Show($"Transaksi Berhasil!\nStruk disimpan di folder project dengan nama:\n{namaFile}", "Sukses", MessageBoxButtons.OK, M
}
catch (Exception ex)
{
    MessageBox.Show($"Gagal menyimpan struk: {ex.Message}");
}
```

Penerapan terakhir ini terletak di struk generator. Hubungan dengan singleton pattern adalah jika user memasukkan inputan diskon dan pajak baru dan menekan button ubah config, maka diskon dan pajak yang ada di Runtime Config yang berstatus sebagai singleton pattern akan mengalami perubahan sesuai dengan inputan dari user.

Dan ketika user menekan tombol checkout, maka program akan melakukan pencetakan struk yang di dalamnya terdapat data pajak dan diskon yang diambil melalui Runtime Config.

2. Utility Pattern

Logika untuk merangkai teks struk pembayaran sangat panjang dan kompleks. Jika logika pemformatan *string* ini ditulis dan digabungkan langsung di dalam kelas antarmuka pengguna (FormTransaksi.cs), akan terjadi pembengkakan kode (*code bloat*). Hal ini melanggar prinsip *Single Responsibility* (Satu Kelas Satu Tanggung Jawab) dan membuat fitur cetak struk tidak bisa digunakan ulang (*reuse*) oleh *form* lain di masa depan.

Keuntungan penerapan:

- **Separation of Concerns:** Memisahkan logika antarmuka (UI) dengan logika pemformatan data, sehingga kode lebih bersih dan mudah di-*maintenance*.
- **Code Reuse:** Fungsi pencetakan struk menjadi modular dan siap dipanggil dari menu mana saja (misalnya menu Riwayat Transaksi) tanpa perlu menulis ulang kode.
- **Stateless Execution:** Tidak memerlukan pembuatan instansiasi objek baru (`new StrukGenerator()`), sehingga menghemat alokasi memori.

```
1 reference
public static class StrukGenerator
{
    1 reference
    public static void GenerateStruk(List<ItemTransaksi> keranjang, decimal subtotal, decimal pajak, decimal diskon, decimal totalAkhir, decimal bayar, decimal kembalian)
    {
        StringBuilder sb = new StringBuilder();
    }
}
```

Pada potongan kode di atas, StrukGenerator dideklarasikan sebagai static class, menjadikannya murni sebagai *Utility Class* atau *Helper*. Sebagai tambahan untuk mengoptimalkan performa, di dalam kelas utilitas ini diterapkan konsep Builder Pattern menggunakan bawaan C# yaitu StringBuilder. Penggunaan StringBuilder sangat efisien untuk menyusun *string* teks struk yang panjang selangkah demi selangkah secara dinamis, dibandingkan dengan penggabungan *string* biasa yang sangat membebani memori.

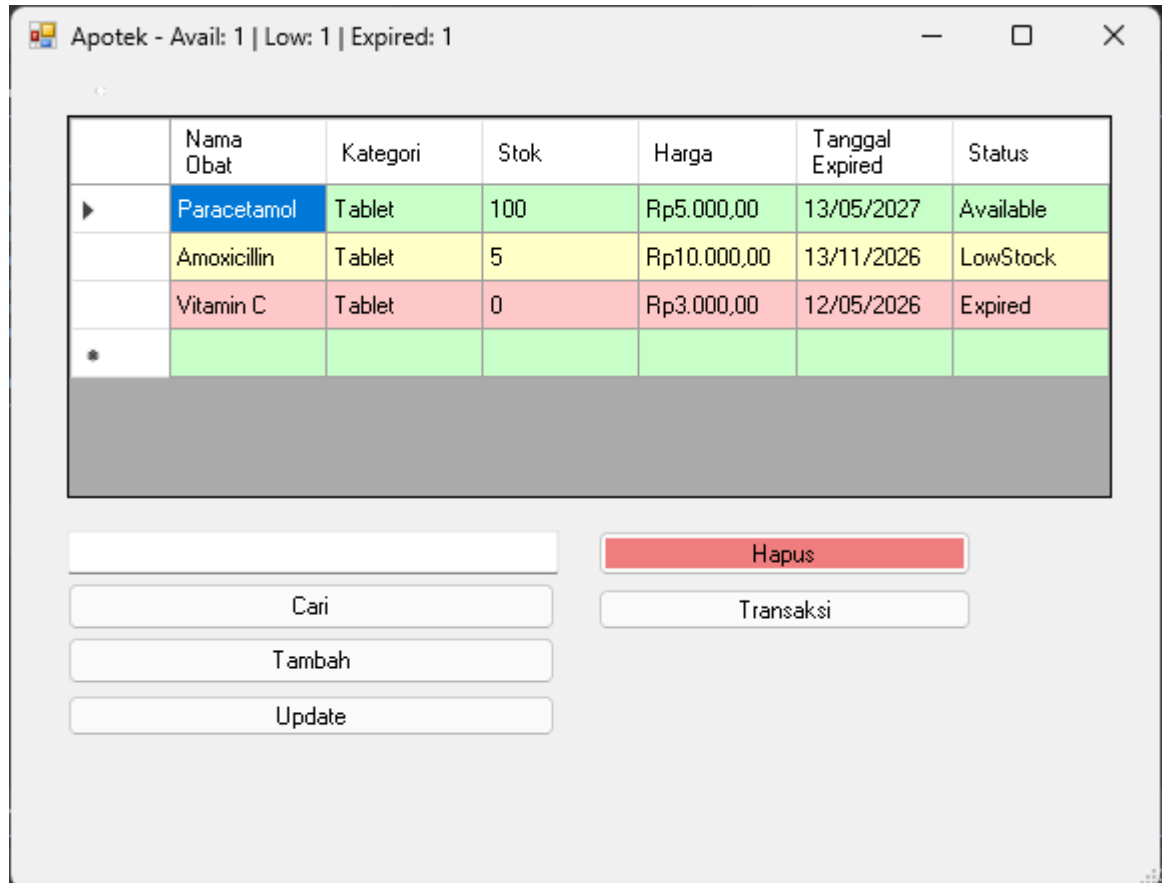
```
// 5. Panggil CODE REUSE (Cetak Struk)
StrukGenerator.GenerateStruk(keranjang, subtotal, RuntimeConfig.PajakPPN, RuntimeConfig.DiskonAktif, grandTotal, uangBayar, kembalian);
```

Penerapan *Utility Class* ini terlihat sangat jelas pada logika penyelesaian transaksi di FormTransaksi. Daripada menuliskan puluhan baris logika pemformatan struk, program hanya cukup memanggil satu baris *method* statis GenerateStruk dari *Utility Class*. Program cukup "melemparkan" parameter data yang dibutuhkan (seperti keranjang obat, nilai pajak, diskon, dan kembalian), kemudian kelas utilitas akan mengambil alih tugas perakitan teks dan mengeksponnya menjadi file dokumen teks (.txt). Pendekatan ini membuat kelas FormTransaksi tetap ringkas dan berfokus pada interaksi pengguna saja.

E. Kontribusi Individu Anggota

1. Muhammad Ayondi Ramadhan

- a. Penerapan Teknik Table Driven dalam Table-Driven Design. Secara sederhana, teknik ini memisahkan "logika aturan" dari "logika eksekusi". banyak pernyataan if-else atau switch-case yang bertumpuk (bercabang-cabang), semua aturan disimpan dalam sebuah struktur data (tabel/dictionary).



Threshold Table (LOW_STOCK_THRESHOLD): Berfungsi sebagai satu

```
/// TABLE 1: THRESHOLD CONFIGURATION
public const int LOW_STOCK_THRESHOLD = 5;

/// TABLE 2: STATUS COLOR MAPPING
/// Format: { "StatusName", Color }
/// Menentukan warna untuk setiap status di UI
private static readonly Dictionary<string, Color> StatusColorTable = new Dictionary<string, Color>()
{
    { "Available", Color.FromArgb(200, 255, 200) },
    { "LowStock", Color.FromArgb(255, 255, 200) },
    { "Expired", Color.FromArgb(255, 200, 200) }
};

/// TABLE 3: STATUS ENUM MAPPING
/// Format: { "lowercase_status", StatusObat.Enum }
/// Untuk convert string API response + enum
private static readonly Dictionary<string, StatusObat> StatusEnumTable = new Dictionary<string, StatusObat>()
{
    { "available", StatusObat.Available },
    { "lowstock", StatusObat.LowStock },
    { "expired", StatusObat.Expired }
};
```

titik pusat untuk menentukan kapan sebuah obat dianggap "stok menipis". Dalam kode ini, batasnya adalah 5.

Color Mapping Table (StatusColorTable): Tabel ini menghubungkan nama status dengan warna visual. Misalnya, status "Expired" dipasangkan dengan warna merah (RGB: 255, 200, 200). Ini mempermudah UI (User Interface) untuk memberikan penanda visual secara otomatis.

Enum Mapping Table (StatusEnumTable): Digunakan untuk menyeragamkan input data dari luar (misal dari database atau API) yang berbentuk teks agar bisa dibaca sebagai tipe data internal sistem (StatusObat).

Rule Table (StatusRuleTable): Ini adalah inti dari sistem. Tabel ini menyimpan urutan prioritas pengecekan. Sistem akan mengecek apakah

```
public static string CalculateStatus(int stok, DateTime expiredDate)
{
    // Sort rules by priority (terendah = pertama di-check)
    var sortedRules = new List<(string StatusName, Func<int, DateTime, bool> Rule, int Priority)>(StatusRuleTable);
    sortedRules.Sort((a, b) => a.Priority.CompareTo(b.Priority));

    // Check setiap rule sampai ada yang match
    foreach (var (statusName, rule, priority) in sortedRules)
    {
        if (rule(stok, expiredDate))
            return statusName;
    }

    // Fallback (seharusnya tidak pernah terjadi karena "Available" selalu true)
    return "Available";
}
```

obat sudah kadaluwarsa dulu (Prioritas 1), kemudian mengecek stok (Prioritas 2), baru terakhir menyatakan tersedia (Prioritas 3).

Bagian fungsi CalculateStatus bekerja dengan cara yang sangat logis bagi pemula:

Sistem mengambil daftar aturan dari StatusRuleTable.

Aturan diurutkan berdasarkan angka Priority. Hal ini penting karena sebuah obat bisa saja stoknya tinggal 2 (Low Stock) tapi juga sudah tanggal kadaluwarsa (Expired). Karena "Expired" punya prioritas lebih tinggi (angka 1), maka sistem akan melabelinya sebagai "Expired".

Sistem menjalankan fungsi pengecekan satu per satu sampai menemukan kondisi yang benar (true), lalu langsung memberikan hasilnya.

lewat


```

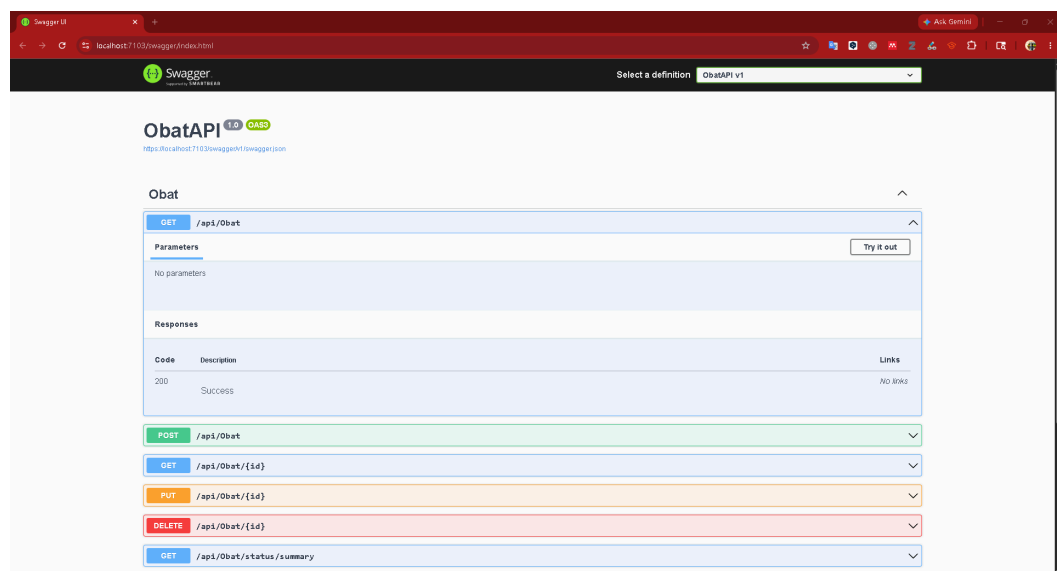
2 references ✓ 10 / 1 passing
public static void GetStatusSummary(List<Obat> obatList, out int available, out int lowStock, out int expired)
{
    available = 0;
    lowStock = 0;
    expired = 0;

    if (obatList == null) return;

    foreach (var obat in obatList)
    {
        string status = CalculateStatus(obat.Stok, obat.ExpiredDate);

        if (status == "Expired")
            expired++;
        else if (status == "LowStock")
            lowStock++;
        else
            available++;
    }
}
}

```



- b. Penerapan API atau Application Programming Interface adalah sebuah "jembatan" atau "penerjemah" yang memungkinkan dua aplikasi atau sistem yang berbeda untuk saling berkomunikasi dan bertukar data.

1. Apa itu ObatApiClient?

Dalam arsitektur aplikasi modern, data biasanya tidak disimpan langsung di dalam aplikasi client, melainkan di server pusat. Kelas ObatApiClient ini bertugas sebagai "kurir" atau jembatan yang menghubungkan aplikasi (Client) dengan server (API) menggunakan protokol HTTP.

2. Fitur Utama dan Cara Kerjanya

A. Manajemen Koneksi (HttpClient)

Di bagian Constructor, kode ini menyiapkan HttpClient. Ada bagian unik di sini:

Validation Callback: Kode ini mengizinkan koneksi ke server meskipun sertifikat SSL-nya tidak valid (sering terjadi saat tahap pengembangan/localhost).

Timeout: Diberi batas waktu 10 detik. Jika server tidak merespons dalam 10 detik, aplikasi akan membatalkan permintaan agar tidak hang (macet).

B. Operasi CRUD Asinkron (Async/Await)

Semua fungsi menggunakan kata kunci async dan Task. Ini sangat penting bagi mahasiswa pemula untuk dipahami:

Mengambil data dari internet butuh waktu. Dengan async, aplikasi tidak akan "membeku" saat menunggu data dari server. User masih bisa mengklik tombol lain sementara data sedang di-download di latar belakang.

C. Penanganan Error yang Informatif

Kode ini tidak hanya menangkap error, tapi juga memberikan pesan yang jelas.

Jika internet mati, muncul pesan "Connection Error".

Jika server lama merespons, muncul pesan "Timeout".

Jika data tidak ditemukan, muncul pesan "ID tidak ditemukan".

Ini membantu pengembang (kita) tahu persis di mana letak kerusakannya.

3. Teknik Khusus: Manual JSON Parsing

Biasanya, programmer menggunakan library otomatis seperti Newtonsoft.Json. Namun, di kode ini, proses baca-tulis data JSON dilakukan secara manual (menggunakan fungsi ParseObatObject dan SerializeObat).

Mengapa ini dilakukan?

Kompatibilitas: Kadang versi .NET lama (4.7.2) sulit menggunakan library modern, jadi kita membuat "parser" sendiri.

Kontrol Penuh: Kita bisa mengatur dengan detail bagaimana data teks dari server diubah menjadi objek Obat di aplikasi kita, termasuk

menangani format tanggal yang berbeda-beda.

4. Alur Kerja Pengolahan Data

Jika kita melihat fungsi GetAllObatAsync, alurnya sangat rapi:

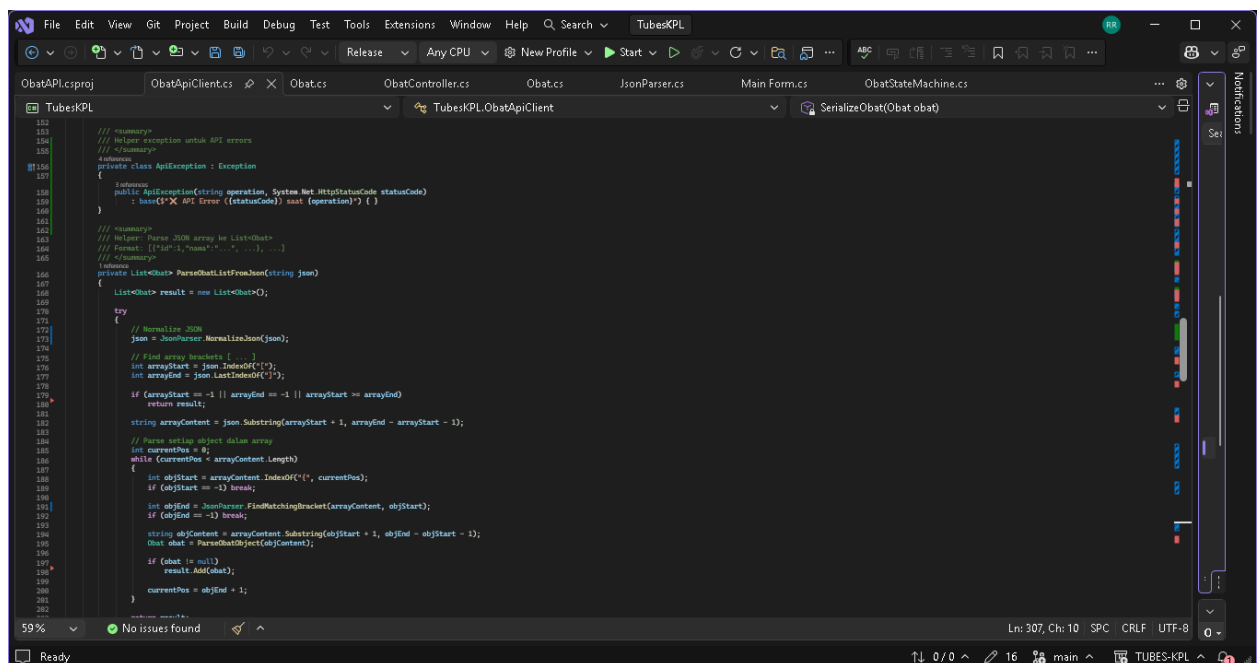
Request: Memanggil URL API.

Check: Memastikan server menjawab dengan sukses (Status Code 200 OK).

Read: Membaca data kiriman server yang berupa teks panjang (JSON).

Parse: Memotong-motong teks tersebut dan memasukkannya ke dalam daftar (List) objek Obat.

Return: Data siap ditampilkan di tabel atau UI aplikasi.



```
152
153 // Summary
154 // Helper exception untuk API errors
155 // Summary
156
157 private class ApiException : Exception
158 {
159     public ApiException(string operation, System.Net.HttpStatusCode statusCode)
160     {
161         base($"API Error ({statusCode}) saat {operation}") { }
162     }
163 }
164
165 // Summary
166 // Helper: Parse JSON array ke List<Obat>
167 // Format: [{"id":1,"nama":"...", ...}, ...]
168 // Summary
169
170 private List<Obat> ParseObatListFromJson(string json)
171 {
172     List<Obat> result = new List<Obat>();
173
174     try
175     {
176         // Normalize JSON
177         json = JsonParser.NormalizeJson(json);
178
179         // Find array brackets [ ... ]
180         int arrayStart = json.IndexOf("[");
181         int arrayEnd = json.LastIndexOf("]");
182
183         if (arrayStart == -1 || arrayEnd == -1 || arrayStart >= arrayEnd)
184             return result;
185
186         string arrayContent = json.Substring(arrayStart + 1, arrayEnd - arrayStart - 1);
187
188         // Parse setiap object dalam array
189         int currentPos = 0;
190         while (currentPos < arrayContent.Length)
191         {
192             int objStart = arrayContent.IndexOf("{", currentPos);
193             if (objStart == -1) break;
194
195             int objEnd = JsonParser.FindMatchingBracket(arrayContent, objStart);
196             if (objEnd == -1) break;
197
198             string objContent = arrayContent.Substring(objStart + 1, objEnd - objStart - 1);
199             Obat obat = ParseObatObject(objContent);
200
201             if (obat != null)
202                 result.Add(obat);
203
204             currentPos = objEnd + 1;
205         }
206     }
207 }
```

2. Raditya Prasna Putra Dharmawan

a. Penerapan Teknik Generics

Teknik *generics/parameterization* diterapkan pada `GenericHelper.cs`, yang memungkinkan definisi *method* dengan berbagai tipe. Salah satu penerapannya pada project aplikasi apotek ini adalah otomatisasi pengisian elemen UI :

```
public static class GenericHelper
{
    1 reference
    public static void GenericComboBox<T>(ComboBox cmb) where T : Enum
    {
        cmb.DataSource = Enum.GetValues(typeof(T));
    }
}
```

Method `GenericComboBox` adalah *method generics* dengan `T` sebagai tipenya. Terdapat pembatasan (`where T : Enum`) agar *method* ini hanya menerima tipe data Enumerasi. Penerapan teknik generic membantu dalam memberikan fleksibilitas. Hasilnya, logika iterasi data cukup ditulis sekali untuk dipakai di mana saja sehingga kode lebih efisien dan mengurangi risiko kesalahan. Pada saat *runtime*, pemanggilan `GenericHelper.GenericComboBox<KategoriObat>(comboBox1)` akan otomatis mengekstrak seluruh nilai kategori (Tablet, Sirup, dll) tanpa perlu di-*hardcode* satu per satu.

b. Penerapan Teknik Code Reuse

Intinya adalah membuat satu kode yang bisa dipakai berulang kali di banyak tempat. Ini membuat program lebih rapi, konsisten, dan jika ada perbaikan, cukup di satu tempat saja.

1. Code Reuse pada Validasi Form

Teknik *code reuse* diterapkan pada class `ObatValidator.cs`, yang berfungsi untuk memeriksa validitas input seperti nama dan nominal stok.

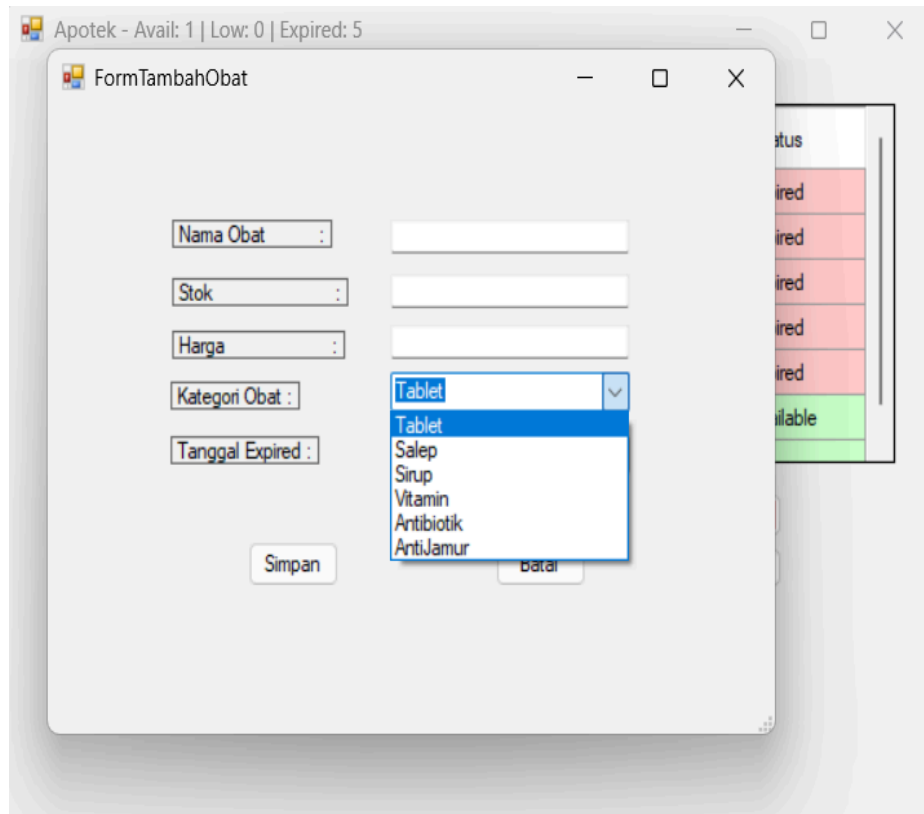
```

public static class ObatValidator
{
    1 reference
    public static bool IsValidInput(string nama, int stok)
    {
        if (string.IsNullOrEmpty(nama)) return false;
        if (stok < 0) return false;
        return true;
    }
}

```

Dengan menerapkan *code reuse*, logika validasi dipisahkan ke dalam kelas tersendiri agar tidak perlu ditulis ulang setiap kali dibutuhkan (misalnya saat klik tombol simpan), sehingga dapat mempermudah proses pemeliharaan.

c. GUI Fitur Tambah



3. Nandana Atha Balangga

a. Penerapan Teknik Runtime Configuration

```
using System;

namespace TubesKPL
{
    5 references
    public static class RuntimeConfig
    {
        3 references
        public static decimal PajakPPN { get; set; } = 0.11m;

        3 references
        public static decimal DiskonAktif { get; set; } = 0.00m;

        1 reference
        public static void UpdateConfig(decimal pajakBaru, decimal diskonBaru)
        {
            PajakPPN = pajakBaru;
            DiskonAktif = diskonBaru;
        }
    }
}
```

Teknik runtime configuration digunakan pada sistem transaksi kasir dengan memanfaatkan class RuntimeConfig. Class ini bertugas untuk membaca dan memuat nilai persentase Pajak dan Diskon saat aplikasi berjalan. Dengan menerapkan teknik ini, jika sewaktu-waktu terdapat perubahan kebijakan (misalnya PPN naik menjadi 12% atau ada diskon promo baru), kita cukup mengubah angkanya pada file konfigurasi tanpa perlu menyentuh kode program utama atau melakukan kompilasi ulang. Hal ini membuat aplikasi menjadi jauh lebih fleksibel dan adaptif terhadap perubahan aturan bisnis

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Windows.Forms;

namespace TubesKPL
{
    1 reference
    public static class StrukGenerator
    {
        1 reference
        public static void GenerateStruk(List<ItemTransaksi> keranjang, decimal subtotal, decimal pajak, decimal diskon, decimal totalAkhir, decimal bayar, decimal kembalian)
        {
            StringBuilder sb = new StringBuilder();

            sb.AppendLine("=====");
            sb.AppendLine("          APOTEK TUBES KPL          ");
            sb.AppendLine("=====");
            sb.AppendLine($"Tanggal : {DateTime.Now:dd/MM/yyyy HH:mm:ss}");
            sb.AppendLine("=====");

            foreach (var item in keranjang)
            {
                sb.AppendLine(item.obat.nama);
                sb.AppendLine($"{item.jumlah} x Rp {item.obat.harga:N0} = Rp {item.Subtotal(N0):N0}");
            }

            sb.AppendLine("=====");
            sb.AppendLine($"Subtotal          : Rp {subtotal:N0}");

            decimal nominalDiskon = subtotal * diskon;
            decimal nominalPajak = (subtotal - nominalDiskon) * pajak;

            sb.AppendLine($"Diskon ({(diskon * 100):0}%)      : - Rp {nominalDiskon:N0}");
            sb.AppendLine($"Pajak PPN ({(pajak * 100):0}%) : + Rp {nominalPajak:N0}");
            sb.AppendLine("=====");
            sb.AppendLine($"TOTAL BAYAR          : Rp {totalAkhir:N0}");
            sb.AppendLine($"UANG DITERIMA        : Rp {bayar:N0}");
            sb.AppendLine($"KEMBALIAN            : Rp {kembalian:N0}");
            sb.AppendLine("=====");
            sb.AppendLine("          TERIMA KASIH, SEMOGA LEKAS SEMBUH          ");
            sb.AppendLine("=====");

            try
            {
                string namaFile = $"Struk_{DateTime.Now:yyyyMMdd_HH:mm:ss}.txt";
                File.WriteAllText(namaFile, sb.ToString());
                MessageBox.Show($"Transaksi Berhasil!\nStruk disimpan di folder project dengan nama:\n{namaFile}", "Sukses", MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Gagal menyimpan struk: {ex.Message}");
            }
        }
    }
}

```

b. Penerapan Teknik Code Reuse

Intinya adalah memisahkan sebuah logika kode agar bisa dipakai berulang kali di banyak tempat dan membuat program lebih rapi. Pada modul transaksi, teknik Code Reuse diterapkan dengan memisahkan logika pembuatan format struk belanja ke dalam class tersendiri, yaitu StrukGenerator.cs. Dengan pendekatan ini, form utama (FormTransaksi) tidak dibebani oleh logika format teks yang panjang. Jika ke depannya fitur cetak struk ini dibutuhkan di halaman lain (misalnya halaman riwayat transaksi), kita cukup memanggil method dari class StrukGenerator ini tanpa harus menulis ulang logika pemformatannya dari nol

c. GUI Fitur Config pajak dan diskon

The screenshot shows a window titled 'Form3' with a standard Windows-style title bar. The interface is divided into several sections. On the left, there are three input fields: 'Cari Obat' with a dropdown arrow, 'Jumlah Obat', and 'Uang Tunai'. Below these are two buttons: 'Tambah Pesanan' and 'Checkout'. In the center-right, there is a large rectangular area labeled 'Keranjang' (Basket) which is currently empty. To the right of the basket, the text 'Total: Rp.0' is displayed. Below this, there are two labels: 'Pajak % :' and 'Diskon % :', each followed by an empty input field. A button labeled 'Ubah Config' (Change Config) is positioned to the right of the 'Total' and 'Pajak' fields. The window has standard minimize, maximize, and close buttons in the top right corner.

Tampilan Konfigurasi Diskon dan Pajak

Sistem menampilkan rincian persentase Diskon dan Pajak secara transparan pada GUI transaksi. Nilai yang ditampilkan pada antarmuka ini terintegrasi langsung dengan teknik Runtime Configuration (config.json), di mana persentase pemotongan atau penambahan harga tidak di-hardcode di dalam form, melainkan dibaca dari file konfigurasi. GUI akan secara otomatis memperbarui kalkulasi Subtotal menjadi Total Akhir berdasarkan konfigurasi Diskon yang sedang aktif saat program dijalankan.

d. GUI Fitur Struk

```
=====
|          | APOTEK TUBES KPL          |
|          |=====
Tanggal : 07/05/2026 22:33:55
|-----
Paracetamol
12 x Rp 5.000 = Rp 60.000
|-----
Subtotal      : Rp 60.000
Diskon (0%)   : - Rp 0
Pajak PPN (11%) : + Rp 6.600
|-----
TOTAL BAYAR   : Rp 66.600
UANG DITERIMA : Rp 100.000.000
KEMBALIAN     : Rp 99.933.400
|-----
|          | TERIMA KASIH, SEMOGA LEKAS SEMBUH |
|          |=====
```

Sebagai langkah akhir dari proses transaksi, GUI akan memunculkan struk digital setelah pembayaran dinyatakan valid dan berhasil. Struk ini digenerate dengan memanfaatkan fungsi Code Reuse dari kelas StrukGenerator. Tampilan struk merangkum seluruh informasi transaksi secara terstruktur, mulai dari daftar obat yang dibeli, perhitungan Subtotal, potongan Diskon, beban Pajak, Total Pembayaran Akhir, hingga jumlah Uang Tunai dan Kembalian. Fitur ini memastikan transparansi dan bukti valid untuk setiap aktivitas kasir.

4. Putra Erlang Duawa

a. Penerapan Teknik Table Driven

Table Driven merupakan sebuah teknik dalam pemrograman yang dimana implementasinya memanfaatkan struktur data seperti array atau tabel untuk menampung dan menggantikan percabangan *if-else* atau *switch-case* dengan tujuan efisiensi code dan tampilan yang lebih ringkas. Dengan penggunaan teknik table driven, eksekusi program bisa ditentukan berdasarkan data yang tersimpan dalam suatu tabel dan bukan dipaparkan dengan suatu kondisi yang eksplisit dan panjang. Kegunaan dari teknik ini bisa meringkas kode sehingga lebih mudah dibaca dan fleksibel terhadap perubahan.

```
2 references
private List<Obat> GetSampleData()
{
    return new List<Obat>()
    {
        new Obat("Paracetamol", 21, 5000, new DateTime(2025, 1, 15), "Tablet"),
        new Obat("Ibuprofen", 12, 7000, new DateTime(2024, 7, 20), "Tablet"),
        new Obat("Sanmol", 15, 3000, new DateTime(2025, 4, 5), "Sirup"),
        new Obat("HRIG", 12, 20000, new DateTime(2024, 3, 10), "AntiJamur"),
        new Obat("Influenza", 10, 2000, new DateTime(2025, 3, 15), "Tablet"),
        new Obat("Vitamin C", 11, 500000, new DateTime(2026, 8, 1), "Vitamin")
    };
}
```

Private List<obat> di sini digunakan untuk menyimpan seluruh data obat yang nantinya akan diolah oleh beberapa method yang telah dibuat oleh kami. Dalam tabel tersebut terdapat data dummy yang berisi nama obat, stok obat, harga, masa berlaku obat, dan juga kategori obat. Nantinya data yang tersimpan dalam tabel tersebut bisa diolah oleh beberapa method, contohnya sebagai berikut:

```
7 references
private void TampilkanData(List<Obat> data)
{
    foreach (var obat in data)
    {
        obat.UpdateStatus();
    }

    DataTable dt = new DataTable();

    dt.Columns.Add("Nama Obat");
    dt.Columns.Add("Kategori");
    dt.Columns.Add("Stok");
    dt.Columns.Add("Harga");
    dt.Columns.Add("Tanggal Expired");
    dt.Columns.Add("Status");

    foreach (var obat in data)
    {
        dt.Rows.Add(
            obat.Nama,
            obat.Kategori,
            obat.Stok,
            obat.Harga.ToString("C"),
            obat.ExpiredDate.ToString("dd/MM/yyyy"),
            obat.Status
        );
    }

    tblObat.DataSource = dt;
}
```

Diatas merupakan sebuah fungsi yang digunakan untuk

menampilkan data obat dengan parameter List<obat> dan akan ditampilkan di tabel GUI.

DataTable dt = new DataTable(); ini merupakan struktur kolom yang nantinya bisa digunakan untuk memunculkan data obat melalui tabel GUI. Dari struktur yang ada di screenshot diatas, terlihat dt menambahkan 6 kolom sesuai dengan kebutuhan List<obat> yaitu nama, kategori, stok, harga, tanggal expired, dan status yang menunjukkan apakah obat sudah expired atau belum.

Kemudian dalam di bagian bawah terdapat foreach(var obat in data) {...}. Syntax ini digunakan untuk melakukan iterasi atau perulangan dan mengolah setiap elemen yang ada didalam suatu koleksi (List<obat>). Nantinya dari syntax tersebut program akan membaca dan mengambil satu obat dan karena setiap obat memiliki properti lain di dalamnya seperti stok, tanggal expired, harga, dan status, maka properti tersebut akan disimpan didalam variables yang tersedia. Kemudian tblObat.DataSource = dt; digunakan untuk menampilkan data ke tabel GUI.

```
private void button1_Click(object sender, EventArgs e)
{
    string inputan = textBox1.Text.ToLower();

    var hasil = daftarObat
        .Where(o => o>Nama.ToLower().Contains(inputan))
        .ToList();

    if (hasil.Count > 0)
    {
        TampilkanData(hasil);
    }
    else
    {
        MessageBox.Show("Obat tidak ditemukan");
    }
}
```

Terakhir, dari screenshot diatas merupakan logika yang digunakan untuk mencari obat berdasarkan nama yang diinputkan oleh user. User akan memasukkan nama dari text box yang tersedia pada GUI. Dari screenshot terlihat terdapat "ToLower" yang mana ini digunakan supaya nama yang diinputkan oleh user otomatis menjadi huruf kecil meskipun kata yang diketik adalah kapital.

Kemudian inputan tadi akan disimpan kedalam variabel yang bernama inputan. var hasil = daftarObat.where(o => o>Nama.ToLower().Contains(inputan)).ToList();

intinya logika ini akan mencari nama obat sesuai dengan nama yang diinputkan oleh user dan sudah disimpan didalam variable inputan. jika nama obat sesuai dan ditemukan, maka akan masuk ke kondisi pertama dan menampilkan nama obat. jika obat tidak ditemukan maka akan muncul pesan "Obat Tidak Ditemukan".

```

private void BtnHapus_Click(object sender, EventArgs e)
{
    int index = TabelKeranjang.CurrentRow?.Index ?? -1;

    var validasi = new List<(bool kondisi, string pesan)>
    {
        (TabelKeranjang.CurrentRow == null, "Pilih item yang mau dihapus dulu ya!"),
        (index < 0 || index >= keranjang.Count, "Item yang dipilih tidak valid!")
    };

    foreach (var aturan in validasi)
    {
        if (aturan.kondisi)
        {
            MessageBox.Show(aturan.pesan);
            return;
        }
    }

    keranjang.RemoveAt(index);
    RefreshKeranjang();
}

```

Contoh lain penerapan Table driven juga terdapat dibalik logika hapus keranjang. Dalam logika tersebut terdapat sebuah index dengan tipe data integer. lalu disebelah kanan terdapat sebuah syntax yang fungsinya untuk memastikan apakah ada kolom yang dipilih dari tabel keranjang?. Jika tidak ada maka akan bernilai null (Index ?? -1). kemudian di bawah ada variabel sementara yang bernama Validasi yang menyimpan sebuah List dengan kondisi dan pesan dengan tipe data boolean dan string. List<Kondisi, Pesan> ini menyimpan sebuah kondisi yang nanti bisa digunakan oleh perulangan foreach.

foreach (var aturan in validasi) ini maksudnya maksudnya adalah melakukan proses perulangan berdasarkan isi dari variabel validasi dan nanti akan disimpan ke variabel aturan. Jika user memilih kolom, maka program akan langsung masuk ke dalam perulangan foreach dan menampilkan pesan sesuai dengan yang ada di dalam koleksi. jika tidak ada error sama sekali, program akan menuju dua index terakhir “keranjang.RemoveAt(index)” yang akan menghapus kolom yang dipilih user. kemudian Refresh Keranjang() merupakan pemanggilan method untuk mengambil kembali isi keranjang yang terbaru.

b. Penerapan Teknik Generic Parameter

Generic parameter merupakan sebuah teknik yang digunakan untuk membuat sebuah struktur data seperti List yang bersifat fleksibel dan tidak tergantung dengan tipe data. Generic parameter itu dilambangkan dengan <T> yang dimaksudkan untuk mendefinisikan tipe data sesuai dengan kebutuhan dan saat objek tersebut ditentukan dan dibuat.

Tujuan dibuat dan penggunaan generic parameter ditunjukkan untuk menciptakan koleksi data yang terstruktur. Dengan terstrukturnya koleksi data sesuai dengan tipe data yang menjadi parameternya, kita sebagai penulis kode hanya perlu memastikan apakah data yang dimasukkan sesuai dengan kebutuhan. Sebagai contoh di bawah ini merupakan potongan kode yang mengimplementasi generic parameter.

```

private void BtnHapus_Click(object sender, EventArgs e)
{
    int index = TabelKeranjang.CurrentRow?.Index ?? -1;

    var validasi = new List<(bool kondisi, string pesan)>
    {
        (TabelKeranjang.CurrentRow == null, "Pilih item yang mau dihapus dulu ya!"),
        (index < 0 || index >= keranjang.Count, "Item yang dipilih tidak valid!")
    };

    foreach (var aturan in validasi)
    {
        if (aturan.kondisi)
        {
            MessageBox.Show(aturan.pesan);
            return;
        }
    }

    keranjang.RemoveAt(index);
    RefreshKeranjang();
}

```

Pada potongan kode diatas, penulis mengimplementasikan teknik generic parameter yaitu `var validasi = new List<bool kondisi, string pesan>`. Ini artinya ada sebuah variabel yang bernama `validasi`, variabel `validasi` ini menyimpan sebuah `List` yang menyimpan kondisi dan pesan dengan tipe data boolean dan string.

Dari sini kita bisa mengetahui kalau `List` yang ada di dalam variabel `validasi` menyimpan dua data yaitu kondisi dan pesan. untuk kondisi, ini akan menyimpan nilai `true` dan `false`. sementara pesan akan merupakan data yang menampilkan pesan.

```

private List<Obat> GetSampleData()
{
    return new List<Obat>()
    {
        new Obat("Paracetamol", 21, 5000, new DateTime(2025, 1, 15), "Tablet"),
        new Obat("Ibuprofen", 12, 7000, new DateTime(2024, 7, 20), "Tablet"),
        new Obat("Sanmol", 15, 3000, new DateTime(2025, 4, 5), "Sirup"),
        new Obat("HRIG", 12, 20000, new DateTime(2024, 3, 10), "AntiJamur"),
        new Obat("Influenza", 10, 2000, new DateTime(2025, 3, 15), "Tablet"),
        new Obat("Vitamin C", 11, 500000, new DateTime(2026, 8, 1), "Vitamin")
    };
}

```

Contoh implementasi lainnya terdapat dalam tabel dummy obat. Dari potongan kode diatas, ada syntax `List<obat>()`. Untuk generic parameternya adalah `obat`, sementara `List` merupakan sebuah wadah atau tempat untuk menampung generic parameternya (`obat`). Sementara untuk `obat` adalah tipe data khusus yang dibuat dengan maksud menyimpan data dari konstruktor class `obat`.

```

public class Obat
{
    // ID unik dari server (untuk identifikasi dan update/delete)
    4 references
    public int Id { get; set; }

    // Nama obat
    14 references
    public string Nama { get; set; } = string.Empty;

    // Kategori obat (string untuk API compatibility)
    7 references
    public string Kategori { get; set; } = "Tablet";

    // Jumlah stok
    9 references
    public int Stok { get; set; }

    // Harga per unit
    7 references
    public decimal Harga { get; set; }

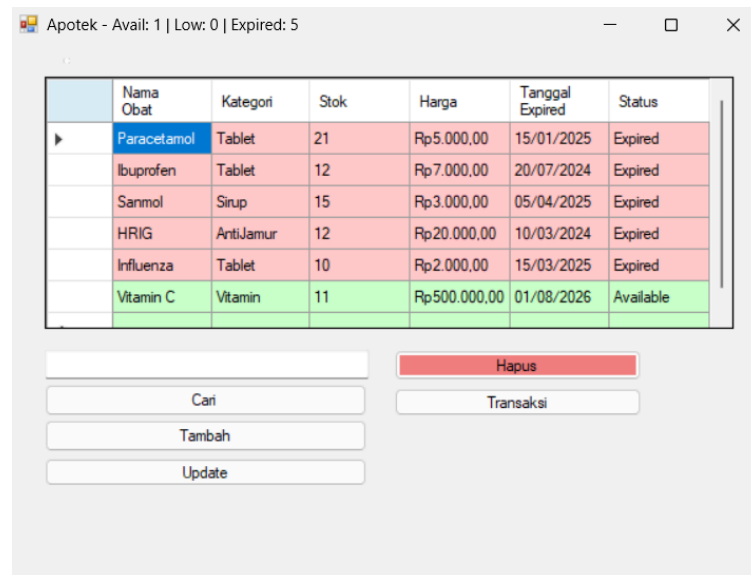
    // Tanggal kadaluarsa
    9 references
    public DateTime ExpiredDate { get; set; }

    // Status obat (Available, LowStock, Expired) - string format dari API
    18 references
    public string Status { get; set; } = "Available";
}

```

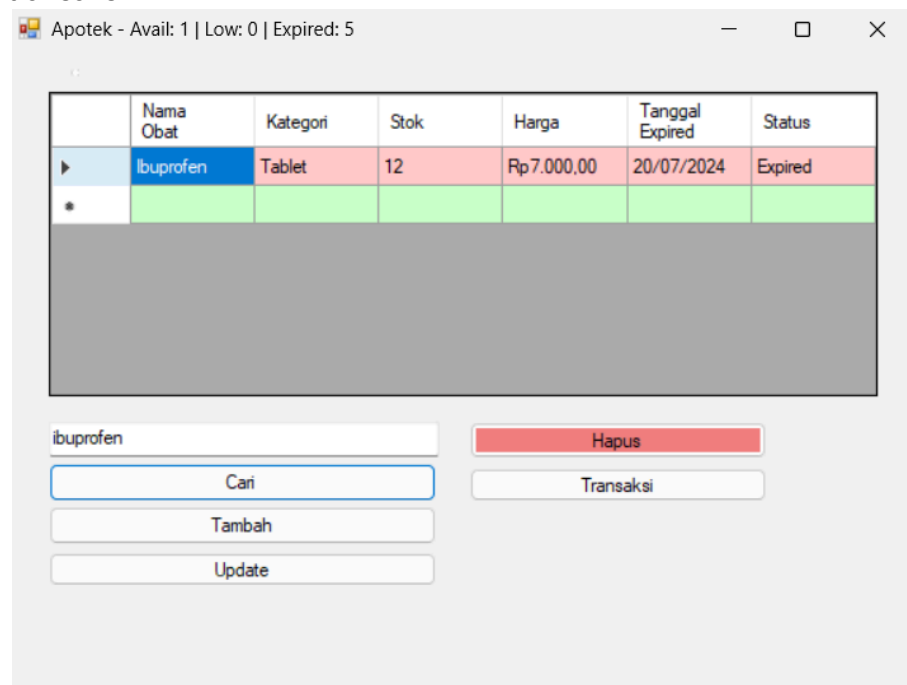
Diatas ini merupakan konstruktor obat yang mana didalamnya menyimpan data seperti id, nama, kategori, stok, harga, expired, dan juga status apakah obat sudah expired atau belum. Class obat ini terhubung dengan List<Obat>, karena Obat menjadi generic parameternya.

c. GUI Fitur Pencarian



Pada gui yang ada di atas, itu merupakan halaman utama atau main dashboard. Berdasarkan penjelasan implementasi table driven dan fitur yang dikerjakan, terdapat text box yang digunakan untuk menginputkan nama obat yang ingin dicari oleh user. Jika obat yang dicari oleh user ada di dalam tabel, maka nanti setelah User menekan tombol cari, nama obat akan muncul di tabel paling atas.

Selain itu, terdapat 4 tombol dengan fungsi yang berbeda yaitu tambah untuk menambahkan obat, Update untuk mengubah data obat, hapus untuk menghapus data obat, dan transaksi untuk melakukan transaksi.



Berikut contoh dari hasil pencarian obat ibuprofen. Ketika obat berhasil ditemukan, maka tabel akan langsung menampilkan obat yang dicari ke daftar teratas.

d. GUI Fitur Transaksi

The screenshot shows a window titled 'Form3' with a light blue border. On the left, there are three input fields: 'Cari Obat' with a dropdown arrow, 'Jumlah Obat', and 'Uang Tunai'. Below these are three buttons: 'Tambah Pesanan', 'Checkout', and 'Ubah Config'. On the right, there is a large grey rectangular area labeled 'Keranjang'. Below the cart area, there are labels for 'Total: Rp.0', 'Pajak %:', and 'Diskon %:', each followed by an input field.

Diatas merupakan potongan foto yang berisi GUI dari fitur transaksi. jika dilihat lebih jauh, GUI tersebut memiliki combobox untuk menampilkan daftar obat yang akan dibeli. lalu di bawah terdapat 2 text box yang digunakan untuk mengisi jumlah obat yang ingin dibeli dan jumlah uang yang akan dibayar.

Kemudian, di bawah terdapat dua tombol yaitu tambah pesanan yang digunakan untuk menyimpan obat yang ingin dibeli ke dalam tabel keranjang. lalu checkout untuk melakukan transaksi.

The screenshot shows the same 'Form3' window. The 'Cari Obat' dropdown now shows 'HRIG'. The 'Jumlah Obat' field contains the number '2'. The 'Uang Tunai' field contains '100000'. The 'Keranjang' table is now populated with one row: 'HRIG' with a price of 'Rp20.000,00' and a quantity of '2'. Below the table, there are buttons for 'Tambah Pesanan', 'Checkout', 'Hapus', and 'Ubah Config'. The 'Total: Rp.0' label remains, along with the 'Pajak %:' and 'Diskon %:' input fields.

	Nama Obat	Harga	Jumlah
▶	HRIG	Rp20.000,00	2
*			

Diatas merupakan contoh dari penggunaan button tambah pesan. Setelah jumlah obat dimasukkan oleh user. user bisa menekan tombol tambah pesanan untuk menyimpan obat. dan jika user menekan tombol hapus, maka obat yang ada di dalam keranjang akan terhapus.

Form3

Cari Obat:

Jumlah Obat:

Uang Tunai:

Keranjang

	Nama Obat	Harga	Jumlah
*			

Total: Rp.0

Pajak % :

Diskon % :

Form3

Cari Obat:

Jumlah Obat:

Uang Tunai:

Keranjang

	Nama Obat	Harga	Jumlah
*	HRIG	Rp20.000,00	2

Total: Rp.0

Pajak % :

Diskon % :

Uang tidak cukup! Total Pembayaran: Rp44.400,00

diatas merupakan potongan foto dari GUI setelah user menghapus obat dari keranjang dan melakukan checkout meskipun kondisi uang tidak cukup.

5. Chrisel Arthalia Putri

a. Penerapan Teknik Defensive Programming (Validasi Input Aman)

Teknik *Defensive Programming* diterapkan pada `Update Form.cs` untuk memastikan bahwa aplikasi tidak akan mengalami *crash* (berhenti paksa) apabila pengguna memasukkan tipe data yang salah saat mengedit data obat

```
private void SimpanData()
{
    try
    {
        editingObat.nama = txtNama.Text.Trim();

        if (int.TryParse(txtStok.Text, out int newStok))
            editingObat.stok = newStok;

        if (decimal.TryParse(txtHarga.Text, out decimal newHarga))
            editingObat.harga = newHarga;

        editingObat.expiredDate = dtpExpired.Value;
        editingObat.UpdateStatus();

        this.DialogResult = DialogResult.OK;
        this.Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Terjadi kesalahan saat menyimpan: " + ex.Message);
    }
}
```

Alih-alih menggunakan `int.Parse()` yang bisa menyebabkan *error* jika *form* diisi dengan huruf, kode ini menggunakan metode `TryParse`. Metode ini bertindak sebagai tameng (defensif) yang akan mengecek terlebih dahulu apakah input dari kotak teks (seperti `txtStok` dan `txtHarga`) benar-benar berisi angka. Jika valid, data baru akan disimpan ke dalam variabel `newStok` atau `newHarga`. Blok kode ini juga dibungkus dalam blok `try-catch` sebagai lapis pertahanan tambahan untuk menangkap *error* yang tidak terduga.

b. Penerapan Teknik Code Reuse

Sama seperti pada bagian form penambahan data, prinsip *Code Reuse* (penggunaan kembali kode) juga diterapkan secara maksimal pada saat memperbarui data obat di `Update Form.cs`.

```
editingObat.expiredDate = dtpExpired.Value;
editingObat.UpdateStatus();

this.DialogResult = DialogResult.OK;
this.Close();
```

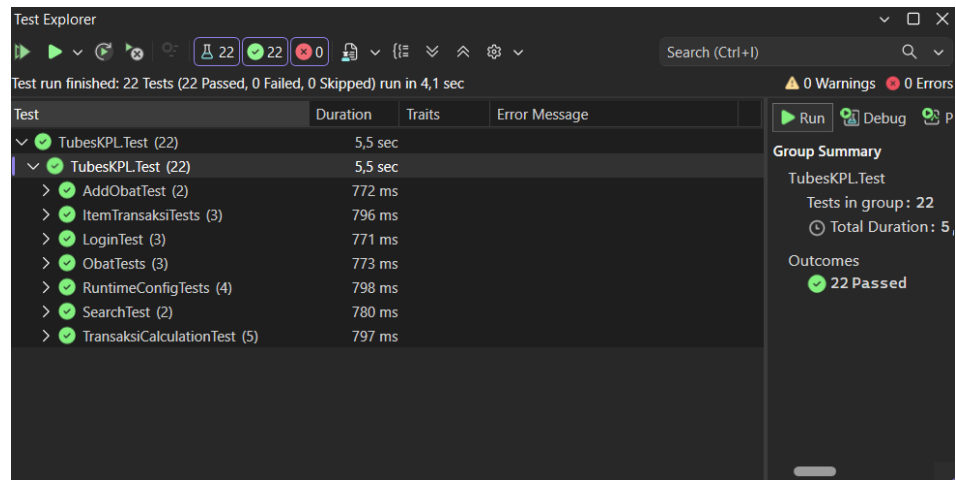
Saat atribut stok atau tanggal kadaluarsa (*expired date*) diubah oleh pengguna, status ketersediaan obat (apakah menjadi `LowStock` atau `Expired`) otomatis bisa berubah. Alih-alih menulis ulang logika pengecekan kondisi `if-else` untuk status tersebut di dalam `Update Form.cs`, pengembang cukup memanggil kembali (*reuse*) fungsi `UpdateStatus()` milik objek `editingObat`. Hal ini menjaga agar kode tetap ringkas, tidak ada duplikasi logika, dan perawatan kode cukup dilakukan di satu tempat (yaitu di kelas `Obat` utama) .

F. Testing

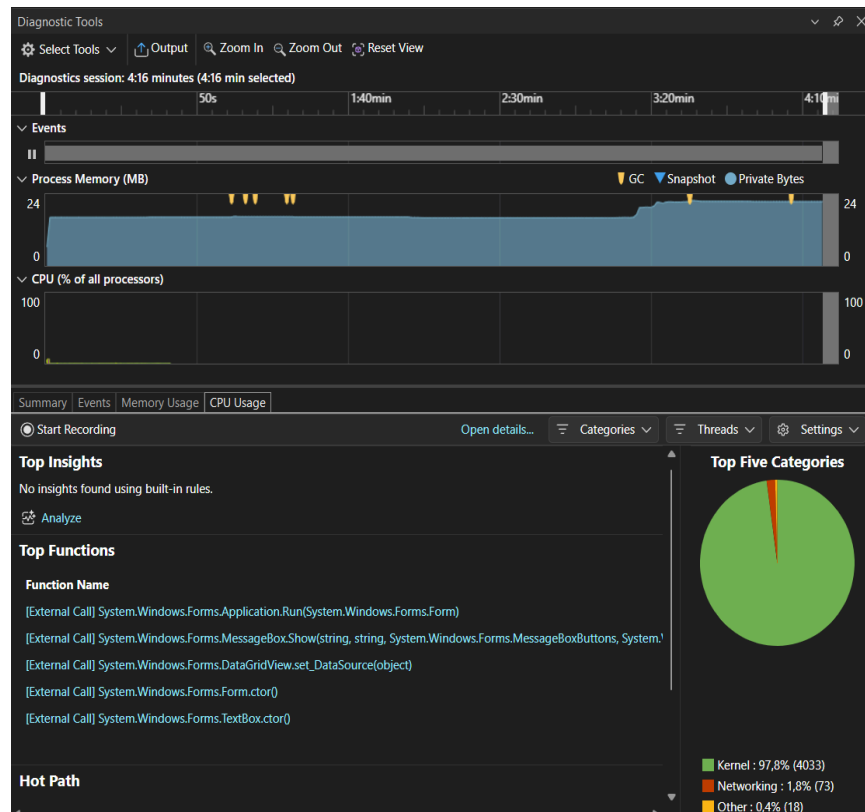
1. Eksekusi Unit Test Logic Main

Kami telah membuat *unit test* untuk masing-masing fungsionalitas aplikasi. Pengujian dibagi ke dalam beberapa modul utama yang mencakup *class* komponen krusial, yaitu: AddObatTest, ItemTransaksiTests, LoginTest, ObatTests, RuntimeConfigTests, SearchTest, dan TransaksiCalculationTest.

Test Case :



Performance Test :



Berdasarkan Coverage :

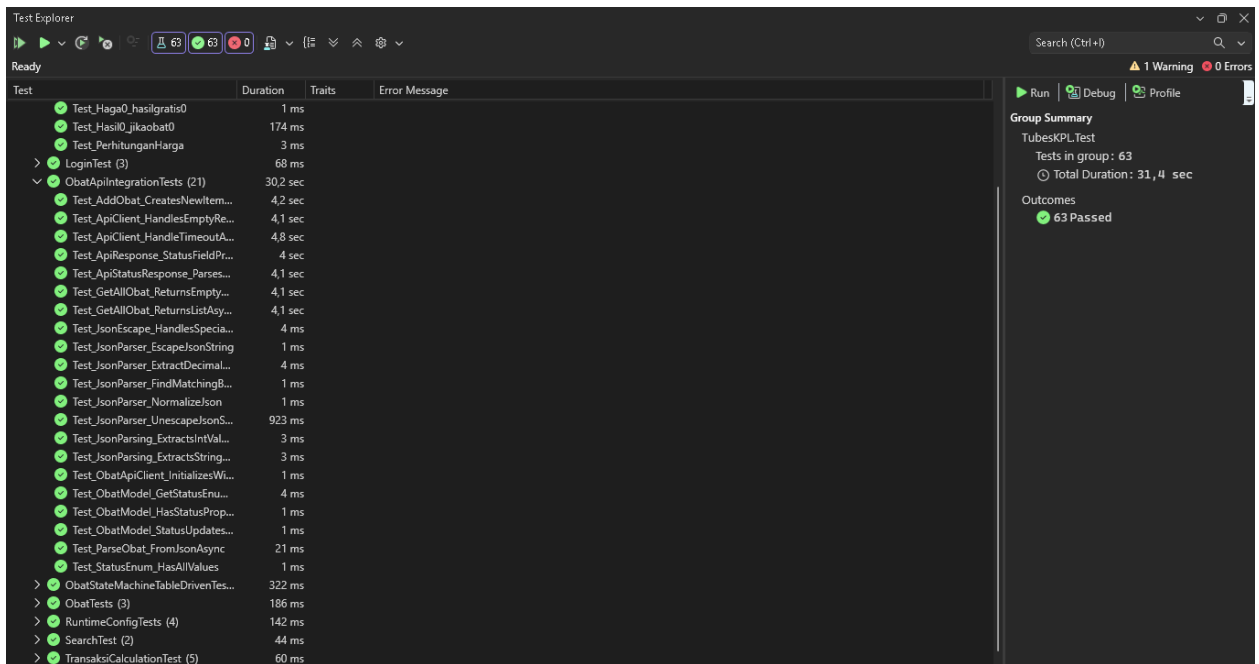
tubeskpl.test.dll	100,0%	100,0%
{ } TubesKPLTest	100,0%	100,0%
AddObatTest	100,0%	100,0%
ItemTransaksiTests	100,0%	100,0%
LoginTest	100,0%	100,0%
ObatTests	100,0%	100,0%
Test_Expired_JadiExpired()	100,0%	100,0%
Test_StokRendah_jadiLowStock()	100,0%	100,0%
Test_StokBanyak_jadiAvailable()	100,0%	100,0%
RuntimeConfigTests	100,0%	100,0%
Test_UbahNilaiPajak_DanDiskon()	100,0%	100,0%
Test_PajakDiskon_Semua0()	100,0%	100,0%
Test_Pajak15_Diskon0()	100,0%	100,0%
Test_Pajak0_Diskon20()	100,0%	100,0%
SearchTest	100,0%	100,0%
TransaksiCalculationTest	100,0%	100,0%
Test_TotalTanpa_PajakDiskon()	100,0%	100,0%
Test_Kembalian_UangPas()	100,0%	100,0%
Test_Kembalian_UangLebih()	100,0%	100,0%
Test_TotalDengan_PajakDiskon()	100,0%	100,0%
Test_UangKurang()	100,0%	100,0%
AddObatTest.<>c	100,0%	100,0%
<Test_ListBerisi_ObatBaru>b_1_0(TubesKPL.Obat)	100,0%	100,0%
SearchTest.<>c__DisplayClass1_0	100,0%	100,0%
<test_Data_Ditemukan>b_0(TubesKPL.Obat)	100,0%	100,0%
SearchTest.<>c__DisplayClass2_0	100,0%	100,0%
<test_Data_TidakDitemukan>b_0(TubesKPL.Obat)	100,0%	100,0%

Seluruh rangkaian pengujian telah berhasil dijalankan berdasarkan fungsionalitas yang ada pada *project* aplikasi apotek dengan ringkasan:

- **Total Tes Dijalankan:** 20
- **Tes Berhasil (Passed):** 20 (Tingkat kelulusan 100%)
- **Tes Gagal (Failed):** 0
- **Tes Dilewati (Skipped):** 0
- **Durasi Eksekusi:** Pengujian inti berjalan dalam waktu yang sangat cepat, yaitu 21 milidetik.

Hasil pengujian ini menunjukkan bahwa seluruh logika bisnis, kalkulasi harga transaksi, validasi *login*, hingga aturan transisi status obat telah berjalan stabil dan reliabel sesuai dengan skenario yang diharapkan.

2. Eksekusi Unit Testing API



The screenshot displays the Visual Studio Test Explorer interface. The left pane shows a tree view of test categories and individual test cases. The middle pane lists the test results in a table with columns for Test, Duration, Traits, and Error Message. The right pane provides a summary of the test group, including the total number of tests, the total duration, and the outcomes.

Test	Duration	Traits	Error Message
Test_Haga0_hasilgratis0	1 ms		
Test_Hasi0_jikaobat0	174 ms		
Test_PerhitunganHarga	3 ms		
LoginTest (3)	68 ms		
ObatApiIntegrationTests (21)	30.2 sec		
Test_AddObat_CreatesNewItem...	4.2 sec		
Test_ApiClient_HandlesEmptyRe...	4.1 sec		
Test_ApiClient_HandleTimeoutA...	4.8 sec		
Test_ApiResponse_StatusFieldPr...	4 sec		
Test_ApiStatusResponse_Parses...	4.1 sec		
Test_GetAllObat_ReturnsEmpty...	4.1 sec		
Test_GetAllObat_ReturnsListAgy...	4.1 sec		
Test_JsonEscape_HandlesSpecia...	4 ms		
Test_JsonParser_EscapeJsonString	1 ms		
Test_JsonParser_ExtractDecimal...	4 ms		
Test_JsonParser_FindMatchingB...	1 ms		
Test_JsonParser_NormalizeJson	1 ms		
Test_JsonParser_UnescapeJsonS...	923 ms		
Test_JsonParsing_ExtractsIntVal...	3 ms		
Test_JsonParsing_ExtractsString...	3 ms		
Test_ObatApiClient_InitializesWi...	1 ms		
Test_ObatModel_GetStatusEnu...	4 ms		
Test_ObatModel_HasStatusProp...	1 ms		
Test_ObatModel_StatusUpdates...	1 ms		
Test_ParseObat_FromJsonAsync	21 ms		
Test_StatusEnum_HasAllValues	1 ms		
ObatStateMachineTableDrivenTes...	322 ms		
ObatTests (3)	186 ms		
RuntimeConfigTests (4)	142 ms		
SearchTest (2)	44 ms		
TransaksiCalculationTest (5)	60 ms		

Group Summary
TubesKPLTest
Tests in group: 63
Total Duration: 31,4 sec

Outcomes
63 Passed

G. Penutup

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa aplikasi Manajemen Apotek berbasis desktop ini berhasil dikembangkan sesuai dengan tujuan utama proyek, yaitu membantu proses pengelolaan data obat, transaksi penjualan, serta monitoring stok secara lebih efektif dan efisien. Sistem yang dibangun telah mampu mendukung berbagai kebutuhan utama apotek, mulai dari fitur login, CRUD data obat, pencarian obat, transaksi kasir, hingga pengelolaan status obat secara otomatis.

Dalam proses pengembangannya, tim juga berhasil menerapkan berbagai teknik konstruksi perangkat lunak seperti Generics, Code Reuse, Runtime Configuration, Table-Driven, Generic Parameter, dan Defensive Programming. Selain itu, penerapan design pattern seperti Singleton Pattern, State Pattern, dan Strategy Pattern membantu menjaga struktur kode agar lebih terorganisir, fleksibel, serta mudah dikembangkan di masa mendatang.

Berdasarkan hasil pengujian unit testing, seluruh fitur utama aplikasi berjalan dengan baik tanpa ditemukan kegagalan pengujian. Seluruh test case berhasil dijalankan dengan tingkat keberhasilan 100%, sehingga menunjukkan bahwa logika bisnis aplikasi telah berjalan secara stabil dan sesuai dengan kebutuhan sistem.

Melalui tugas besar ini, seluruh anggota kelompok juga memperoleh pengalaman dalam bekerja secara kolaboratif dalam pengembangan perangkat lunak, mulai dari tahap analisis kebutuhan, implementasi fitur, hingga proses testing dan dokumentasi. Diharapkan aplikasi ini dapat menjadi dasar pengembangan sistem apotek yang lebih kompleks dan modern di masa depan, misalnya dengan integrasi database online, laporan berbasis cloud, maupun fitur monitoring stok secara real-time.